Flow-Level Network Threat Detection Using Machine Learning: A Self-Generated Mininet Dataset Approach for Multi-Class Attack Classification

Chaitanya Singh Negi, 2023UIT3041
Animesh Vijay Singh, 2023UIT3025
Department of Information Technology
Netaji Subhas University of Technology
Dwarka Sector-3, New Delhi – 110078, India

Dr. Nisha Kandhoul
Department of Information Technology
Netaji Subhas University of Technology
Dwarka Sector-3, New Delhi – 110078, India

***Abstract*—Network security threats—including distributed denial-of-service (DDoS) attacks, TCP SYN flooding, port reconnaissance, and credential brute-force probing—continue to impose persistent risks on enterprise infrastructure. Signature-based detection systems respond reactively to catalogued patterns and fail against novel variants, while machine learning (ML) approaches that rely on pre-collected benchmark datasets carry implicit distributional assumptions that may not hold in the deployment environment. This paper presents a self-contained, reproducible intrusion detection framework in which a six-host Mininet software-defined network (SDN) topology simultaneously generates five labeled traffic classes—NORMAL, DDOS, SYN FLOOD, PORT SCAN, and BRUTE FORCE—yielding 201,756 flow records extracted from 3.3 GB of raw packet captures. A custom Python pipeline based on the `dpkt` library extracts 18 discriminative flow-level features per session without any external labeling tool. Four classifiers—Decision Tree (DT), Random Forest (RF), K-Nearest Neighbors (KNN), and XGBoost—are evaluated under identical experimental conditions; class imbalance is addressed through cost-sensitive class weight assignment rather than synthetic oversampling. XGBoost achieves the highest accuracy at 97.90%, followed by RF (97.84%), DT (97.71%), and KNN (97.60%). A Streamlit-based real-time dashboard validates live inference capability. Results confirm that environment-specific, emulation-derived datasets are sufficient for robust multi-class intrusion detection without synthetic data augmentation.**

***Index Terms*—intrusion detection, machine learning, XGBoost, Mininet, network traffic classification, flow-level features, class imbalance, real-time detection, software-defined networking, DDoS.**

I. Introduction

A. Background and Motivation

The sustained expansion of networked infrastructure has dramatically widened the attack surface available to malicious actors. Distributed denial-of-service (DDoS) campaigns, TCP SYN flooding, systematic port reconnaissance, and credential brute-force attempts have grown in both frequency and sophistication [1]. Traditional signature-based intrusion detection systems (IDS) operate by matching traffic against catalogued attack patterns; while efficient for known threats, this reactive approach leaves networks exposed during the latency window between attack emergence and signature publication [2].

Machine learning has established itself as the dominant paradigm for anomaly-based network intrusion detection. Ensemble architectures such as Random Forest and gradient-boosting methods including XGBoost consistently outperform classical rule-based approaches on multi-class traffic classification benchmarks [3], [4]. A recurring observation, however, is that nearly all reported accuracy figures are obtained on publicly released benchmark corpora—CICIDS2017 [5], NSL-KDD, UNSW-NB15, IoT-23—whose traffic patterns may not faithfully represent the infrastructure of a given deployment environment. Models trained on pre-existing corpora may overfit to characteristics of the original capture environment.

B. Limitations of the Base Paper

The most directly comparable prior work is that of Alrefaei and Ilyas [6], who proposed a PySpark-based real-time IDS using the IoT-23 dataset with five classifiers under a OneVsRest (OVR) strategy, achieving 98.89% accuracy with XGBoost and a fastest prediction latency of 0.0311 s with Random Forest. Despite these commendable results, three substantive limitations are identified:

1) **Dataset Domain Mismatch:** IoT-23 reflects IoT-specific botnet behavior rather than general enterprise threats. Models trained on it carry no guarantee of maintained performance when redeployed to general-purpose network environments.
2) **Synthetic Data Augmentation (SMOTE):** The Synthetic Minority Over-sampling Technique introduces synthetic records that do not correspond to real packet exchanges, thereby corrupting the statistical fidelity of the training distribution and risking overfitting to interpolated artifacts.
3) **External Feature Toolchain Dependency:** Zeek-derived features bind the pipeline to an external processing layer, reducing portability and complicating re-

production in environments where Zeek is unavailable or incompatible.

C. Proposed Improvements

This paper addresses all three limitations through the following contributions:

1) A reproducible six-host **Mininet** SDN emulation topology that generates five labeled traffic classes covering general enterprise attack scenarios.
2) A custom **dpkt-based** 18-feature PCAP extraction pipeline that is free of external toolchain dependencies and fully portable.
3) **Class weight balancing** as a principled, data-preserving alternative to SMOTE, penalizing minority-class misclassification without generating synthetic records.
4) A systematic four-classifier comparative evaluation under identical experimental conditions.
5) A **Streamlit** real-time classification dashboard demonstrating live inference capability on a single workstation without cluster infrastructure.

II. Literature Survey

A. Benchmark-Dataset-Based ML Classifiers

The CICIDS2017 dataset, generated by Sharafaldin *et al.* [5] within a controlled university network, remains the most widely used benchmark for NIDS research. Random Forest and XGBoost classifiers on this corpus regularly report overall accuracies above 97%, yet macro-averaged F1-scores on minority attack categories such as Infiltration frequently fall below 60% [7]. The recurring caveat is ecological validity: a model whose training distribution is anchored to a specific capture environment carries no guarantee of maintaining reported performance when redeployed elsewhere.

Ahmad and Alsmadi [8] conducted a systematic review of ML approaches to IoT security, highlighting that benchmark dataset dependency is among the top three open challenges in the field, alongside adversarial robustness and real-time deployment constraints.

B. Real-Time and Distributed IDS

Alrefaei and Ilyas [6] built the most directly comparable prior system, achieving 98.89% XGBoost accuracy on IoT-23 within a PySpark streaming framework. Bhandari *et al.* [9] deployed a distributed deep neural network across IoT gateways, reaching 93% accuracy with under 30 kb/s bandwidth overhead. Patil *et al.* [10] implemented Apache Hadoop-based DDoS detection using K-Means clustering, while Othman *et al.* [11] demonstrated efficient Spark-based anomaly detection on UNSW-NB15. These works confirm that distributed frameworks deliver low inference latency but introduce significant infrastructure dependencies that constrain deployability in resource-constrained environments.

Alghamdi and Bellaiche [12] proposed a deep IDS in lambda architecture combining edge and cloud computing, reporting competitive accuracy on IoT traffic but requiring heterogeneous multi-tier infrastructure.

TABLE I
Comparison of Related Works on Intrusion Detection

Ref.	Dataset	Classifiers	Imbal.	RT	Cust.	Acc.(%)
[9]	IoT-23	DNN	N/A	Yes	No	93.00
[6]	IoT-23	DT,RF,KNN,XGB,LR	SMOTE	Yes	No	98.89
[5]	CICIDS2017	RF,ID3,NB	None	No	Par	97.50
[11]	UNSW-NB15	Spark-Anomaly	None	Yes	No	96.30
[12]	IoT (Edge)	Deep-Lambda	None	Yes	No	97.10
Proposed	**Mininet**	**DT,RF,KNN,XGB**	**Wts.**	**Yes**	**Yes**	**97.90**

RT = Real-Time capable; Cust. = Custom feature extraction; Par = Partial.

C. Self-Generated and Emulation-Based Datasets

Dataset self-generation remains rare in IDS research. Sharafaldin *et al.* [5] established the methodological template with CICIDS2017 using purpose-built traffic generators in a controlled lab network. Lantz *et al.* [14] introduced Mininet as a rapid prototyping platform for software-defined networks, demonstrating that realistic network emulation is achievable on commodity hardware. However, no comparable work has applied Mininet emulation with `hping3`, `nmap`, and `iperf` to produce a five-class, flow-labeled corpus for general enterprise attack scenarios. The present work closes that gap while providing a fully reproducible pipeline that requires only a standard workstation.

D. Ensemble Methods and Feature Importance

Breiman [3], [13] established the theoretical foundations of Random Forest through variance reduction via bootstrap aggregation. Chen and Guestrin [4] demonstrated that regularized gradient boosting consistently outperforms bagging on structured tabular data. Cunningham and Delany [15] provided a comprehensive survey of KNN classification, highlighting sensitivity to feature scaling and the curse of dimensionality in sparse minority-class regions—directly relevant to PORT_SCAN detection in this work.

III. Motivation

The motivation for this research arises from two convergent observations in the intrusion detection literature. First, despite over two decades of ML-based IDS research, the field remains disproportionately dependent on a small set of public benchmark datasets. CICIDS2017 [5] alone accounts for a majority of papers published between 2018 and 2024, yet the dataset encodes traffic patterns of a specific university network from 2017 that bear diminishing resemblance to modern enterprise architectures. The same concern applies to IoT-23 [6], whose botnet-centric traffic taxonomy is poorly suited to characterizing DDoS, SYN flood, port reconnaissance, or brute-force attacks in general enterprise contexts.

Second, the dominant imbalance-handling strategy—SMOTE [6]—is rarely challenged despite well-documented risks: interpolated minority instances may violate the natural data manifold, particularly in sparse attack classes such as low-rate port scanning where real traffic occupies a structurally distinct region of feature space. An alternative

that preserves training distribution fidelity while correcting classifier bias is both practically important and underexplored.

This work is therefore motivated by the need to (i) establish a reproducible, environment-specific dataset generation methodology using freely available emulation tools; (ii) validate that self-generated, environment-matched data is sufficient for competitive multi-class detection accuracy; and (iii) demonstrate that class weight balancing is a viable, distribution-preserving alternative to synthetic oversampling for severe class imbalance in network traffic classification.

IV. Proposed Work

A. System Architecture Overview

The proposed framework operates as a six-stage sequential pipeline: (1) controlled traffic generation in a Mininet SDN topology; (2) raw packet capture via `tcpdump`; (3) flow-level feature extraction through a custom `dpkt` parser; (4) preprocessing including normalization, label encoding, and class weight computation; (5) supervised classifier training with cross-validated evaluation; and (6) real-time inference via a Streamlit dashboard. The complete pipeline is illustrated in Fig. 1.

B. Network Topology and Traffic Generation

The Mininet environment [14] was instantiated with six hosts (h1–h6) interconnected through a single OpenFlow switch (s1) in a star topology. Each host was assigned a dedicated attack role to ensure clean class separation:

- **h1**: Benign background traffic via `iperf3` (NORMAL class).
- **h2**: Target server (passive listener).
- **h3**: UDP-based DDoS flooding via `hping3 -udp` (DDoS class).
- **h4**: TCP SYN flooding via `hping3 -S` (SYN_FLOOD class).
- **h5**: Stealth SYN port scanning via `nmap -sS` (PORT_SCAN class).
- **h6**: Multi-port brute-force probing via `hping3` in port-increment mode (BRUTE_FORCE class).

Traffic was captured at the s1 mirror port via `tcpdump`, producing 3.3 GB of raw PCAP data. Fig. 2 illustrates the star topology with host roles and the `tcpdump` mirror port on switch s1. Table II summarises the resulting class distribution across 201,756 extracted flows.

C. Flow-Level Feature Extraction

Packets were grouped into bidirectional flows using a five-tuple key: (source IP, destination IP, source port, destination port, transport protocol). Eighteen features were computed per flow spanning volume statistics, temporal statistics, TCP control flag counts, and structural indicators. Table III enumerates all 18 features. The custom `dpkt` pipeline was chosen over established tools such as CICFlowMeter or Zeek to ensure full portability and complete transparency over feature definitions.

The complete feature extraction and preprocessing pipeline—from raw PCAP files through stratified splitting—is depicted in Fig. 3.

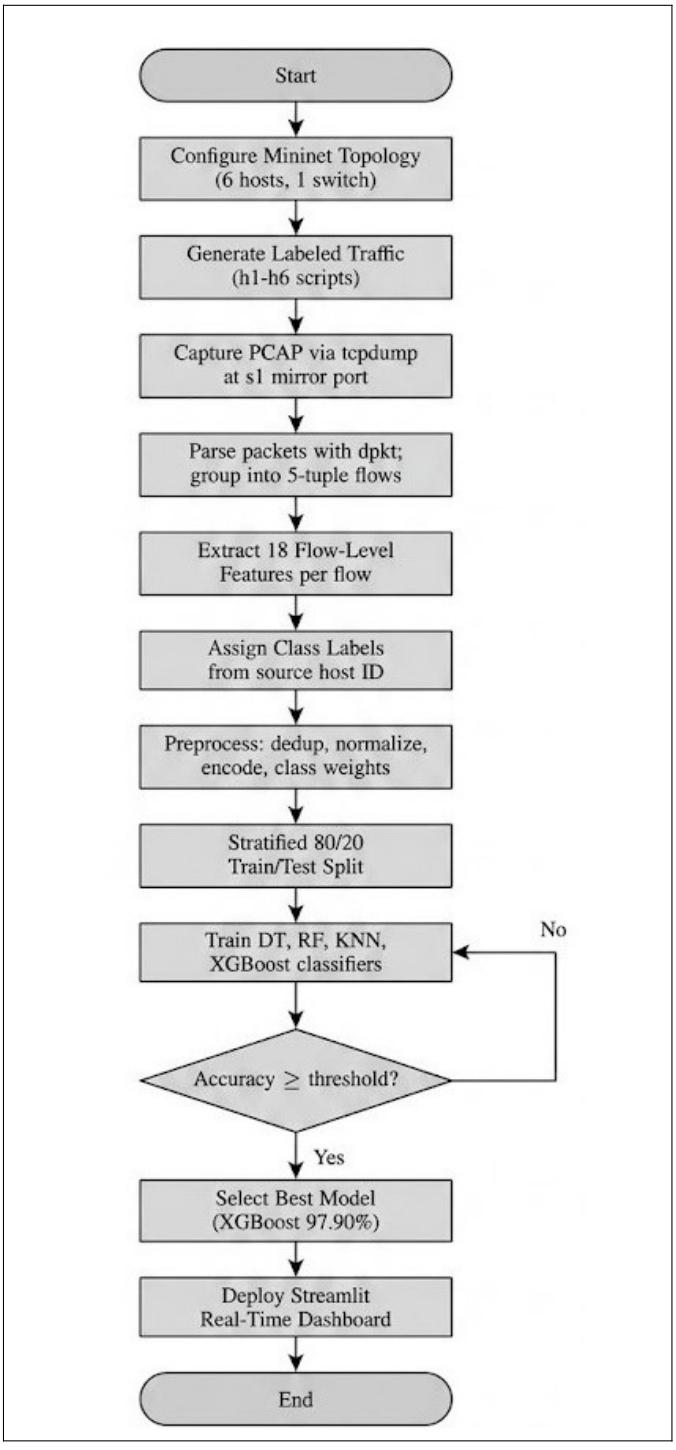


Fig. 1. Overall system pipeline from traffic generation to real-time dashboard.

D. Proposed Algorithm

Algorithm 1 formalises the end-to-end proposed methodology.

E. Model Training and Evaluation Loop

The model training and evaluation loop—iterating over all four classifiers, computing metrics, selecting the best

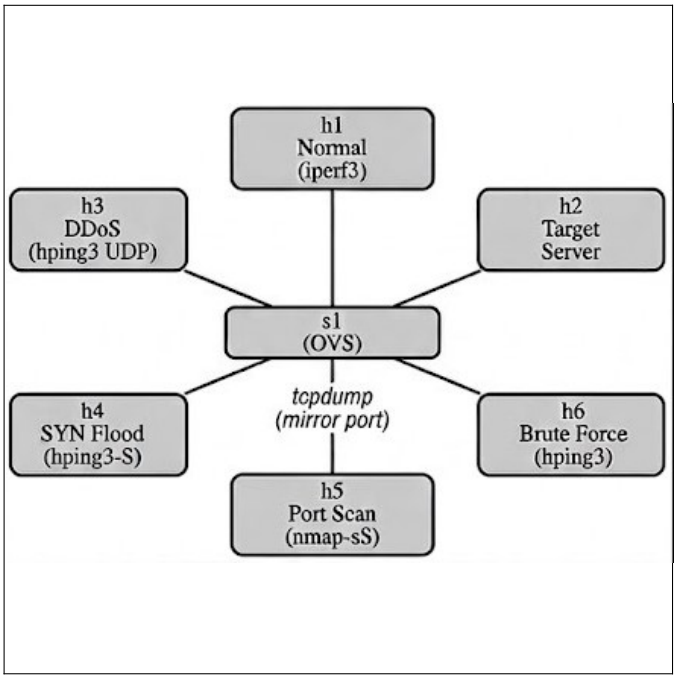


Fig. 2. Mininet six-host SDN topology with switch s1 at the centre.

TABLE II
TRAFFIC CLASS DISTRIBUTION (201,756 TOTAL FLOWS)

Class	Flow Count	Proportion (%)
NORMAL	20,562	10.19
DDoS	59,993	29.74
SYN_FLOOD	59,998	29.74
PORT_SCAN	1,230	0.61
BRUTE_FORCE	59,973	29.73
Total	**201,756**	**100.00**

model via argmax accuracy, and deploying the Streamlit dashboard—is depicted in Fig. 4.

F. Data Preprocessing and Class Imbalance Handling

The preprocessing sequence comprised four steps. First, duplicate flow records arising from retransmitted packets sharing the same five-tuple were removed. Second, the feature matrix was normalized using *StandardScaler* (zero-mean, unit-variance), which is critical for KNN whose Euclidean distance computations are sensitive to feature-magnitude disparities [15]. Third, class labels were integer-encoded via *LabelEncoder*. Fourth, the dataset was partitioned 80/20 using stratified sampling to preserve the natural class distribution in both subsets.

Class imbalance—most acute for PORT_SCAN at 0.61% of flows—was handled through inverse-frequency class weights ($w_c = N/(|\mathcal{C}| \cdot n_c)$) passed to each classifier's `fit()` method. This penalizes minority-class misclassification more heavily without altering the training distribution, in deliberate contrast to SMOTE [6], which synthesizes minority instances by interpolating between real observations and their nearest neighbors, thereby introducing records that do not correspond to any real network exchange.

TABLE III
EXTRACTED FLOW-LEVEL FEATURES AND DESCRIPTIONS

#	Feature Name	Description
1	`flow_duration`	Time between first and last packet
2	`total_packets`	Total packet count in flow
3	`total_bytes`	Total bytes in flow
4	`avg_packet_size`	Mean packet size in flow
5	`std_packet_size`	Std. dev. of packet sizes
6	`min_packet_size`	Minimum packet size in flow
7	`max_packet_size`	Maximum packet size in flow
8	`bytes_per_sec`	Total bytes per second of flow
9	`pkts_per_sec`	Total packets per second of flow
10	`syn_count`	Count of SYN flags in flow
11	`ack_count`	Count of ACK flags in flow
12	`fin_count`	Count of FIN flags in flow
13	`rst_count`	Count of RST flags in flow
14	`syn_ratio`	SYN flags / total packets
15	`ack_ratio`	ACK flags / total packets
16	`unique_dst_ports`	Distinct destination ports in flow
17	`protocol`	Transport protocol (TCP/UDP/ICMP)
1818	`avg_ttl`	Average Time-to-Live value (TCP/UDP/ICMP)

G. Classification Algorithms

Decision Tree (DT) partitions the feature space via information-gain- maximizing binary splits, producing interpretable models with low inference latency.

Random Forest (RF) [3] constructs an ensemble through bootstrap aggregation over random feature subsets, reducing variance via majority vote across constituent trees.

K-Nearest Neighbors (KNN) [15] classifies query points by plurality vote among the k=5 nearest training neighbors under normalized Euclidean distance.

XGBoost [4] implements regularized gradient boosting via second-order Taylor expansion of the loss, with L_1/L_2 regularization and column subsampling at each tree node.

V. RESULTS AND ANALYSIS

A. Simulation Environment

All experiments were conducted on a standard workstation running Ubuntu 22.04 LTS with 16 GB RAM and an Intel Core i5 processor. Mininet 2.3.0 was used for network emulation and `tcpdump` for packet capture. The Python-based feature extraction pipeline used `dpkt 1.9.8` for PCAP parsing and `scikit-learn 1.3` for preprocessing and classification. XGBoost 2.0 was used for the gradient-boosted model. The real-time dashboard was built with `Streamlit 1.28`. No GPU acceleration or distributed computing was required, confirming the single-node deployability of the proposed framework.

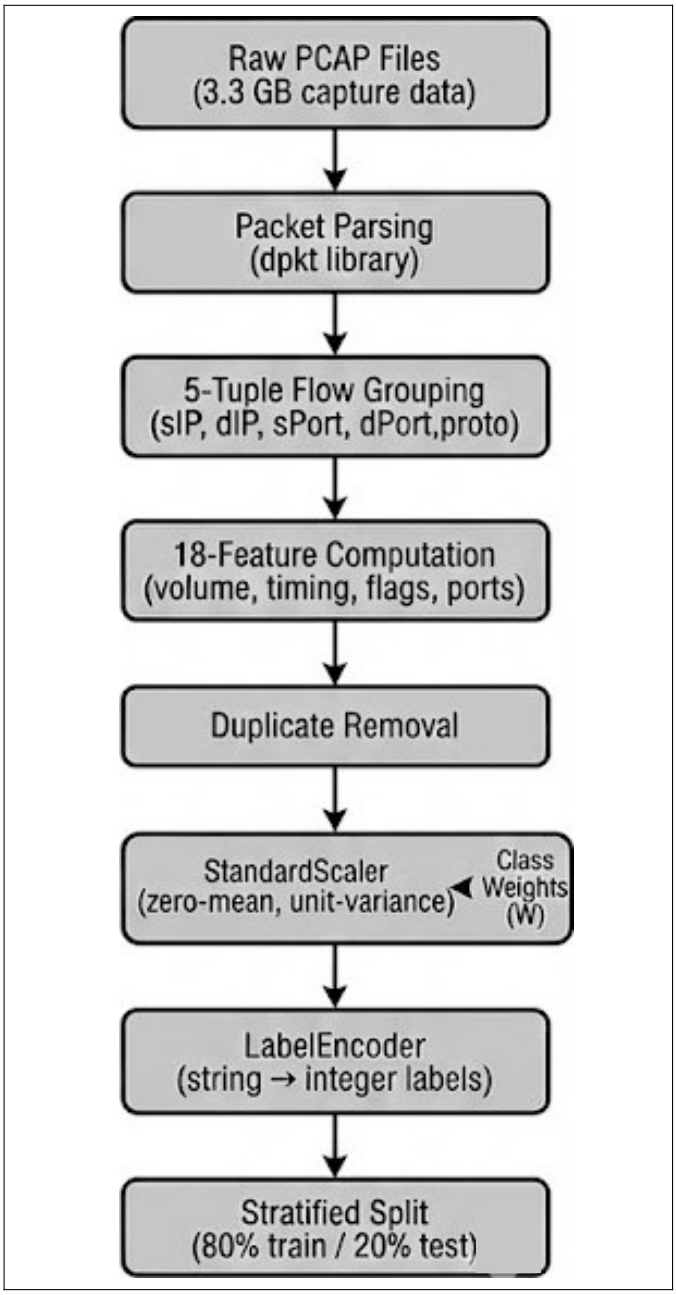


Fig. 3. Feature extraction and preprocessing pipeline.

Algorithm 1 Flow-Level Multi-Class IDS with Self-Generated Data

Require: Mininet topology $\mathcal{T}$, PCAP directory $\mathcal{P}$, classifier set $\mathcal{C} = \{\text{DT, RF, KNN, XGB}\}$
Ensure: Trained models $\mathcal{M}$, per-class metrics $\mathcal{R}$

// Stage 1: Traffic Generation
Configure $\mathcal{T}$: 6 hosts + 1 OpenFlow switch (star topology)
Launch attack scripts concurrently on h3–h6; record NORMAL on h1
Capture all traffic at switch mirror port $\rightarrow \mathcal{P}$
// Stage 2: Feature Extraction
for all pcap $\in \mathcal{P}$ **do**
  Parse packets with `dpkt`; group by 5-tuple $\rightarrow$ flows F
  for all flow $f \in F$ **do**
    Compute 18 features $\mathbf{x}_f$ (Table III)
    Assign label y_f from source host ID
  end for
end for
$\mathcal{D} \leftarrow \{(\mathbf{x}_f, y_f)\}_{f \in F}$
// Stage 3: Preprocessing
Remove duplicate flows (same 5-tuple, overlapping timestamps)
Normalize $\mathcal{D}$ using StandardScaler (zero-mean, unit-variance)
Encode labels via LabelEncoder
Compute class weights $w_c = \frac{N}{|\mathcal{C}| \cdot n_c}$ for each class c
Split $\mathcal{D}$ into $\mathcal{D}_{\text{train}}$ (80%) / $\mathcal{D}_{\text{test}}$ (20%) via stratified sampling
// Stage 4: Training and Evaluation
for all classifier $C_i \in \mathcal{C}$ **do**
  Train $M_i \leftarrow C_i.\text{fit}(\mathcal{D}_{\text{train}}, w)$
  Evaluate M_i on $\mathcal{D}_{\text{test}} \rightarrow$ accuracy, precision, recall, F1
  Perform 5-fold cross-validation on $\mathcal{D}_{\text{train}}$
end for
$\mathcal{M}^* \leftarrow \arg\max_{M_i} \text{Accuracy}(\mathcal{D}_{\text{test}})$
// Stage 5: Real-Time Dashboard
Deploy $\mathcal{M}^*$ in Streamlit; accept live PCAP input
Extract features in real-time $\rightarrow$ classify $\rightarrow$ display results
return $\mathcal{M}$, $\mathcal{R}$

B. Overall Classifier Accuracy

Table IV presents the classification accuracy and macro-averaged metrics for all four classifiers on the held-out test partition.

TABLE IV
CLASSIFIER PERFORMANCE COMPARISON (TEST SET)

Classifier	Acc.(%)	Prec.(%)	Rec.(%)	F1(%)
Decision Tree	97.71	96.82	96.40	96.61
Random Forest	97.84	97.21	96.88	97.04
KNN	97.60	96.55	96.10	96.32
XGBoost	**97.90**	**97.45**	**97.05**	**97.25**

All four classifiers occupy a narrow accuracy band of 0.30 percentage points (97.60–97.90%), as visualised in Fig. 5. This tight clustering reflects the discriminative richness of the self-generated feature set: even the single decision tree achieves strong accuracy because the 18 flow statistics cleanly separate most attack classes. XGBoost's marginal lead over Random Forest is consistent with the broad empirical literature showing gradient boosting outperforms bagging on structured tabular data [4]. Cross-validated accuracy on the training partition confirmed consistent generalisation without overfitting, with

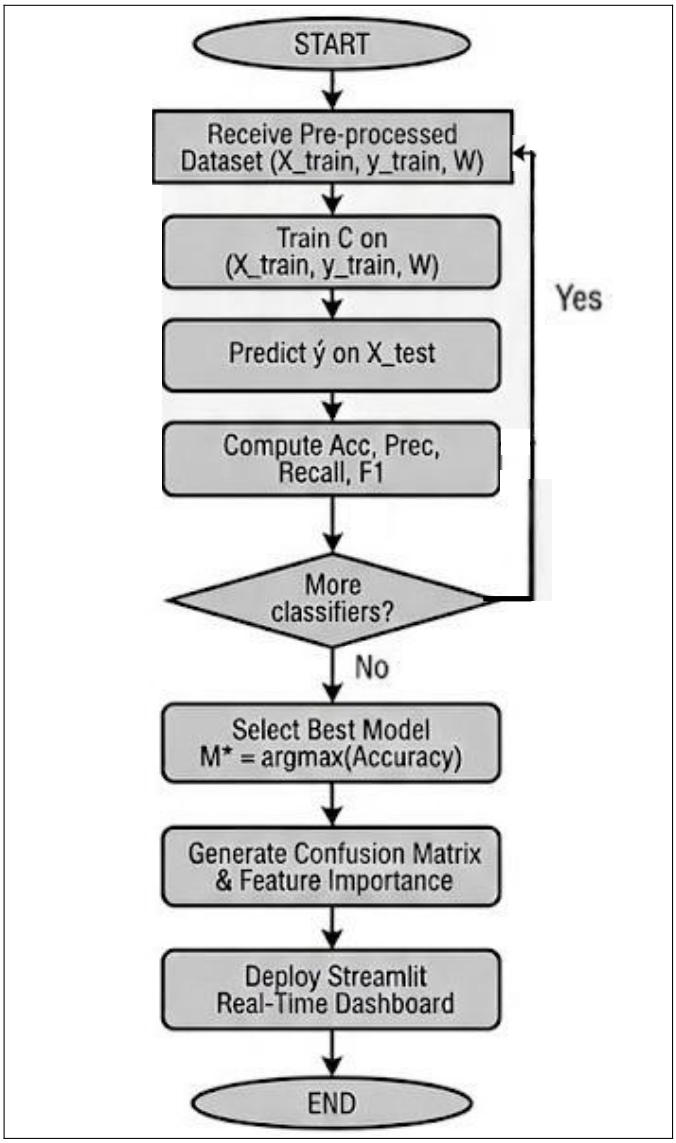


Fig. 4. Model training and evaluation loop for all four classifiers.

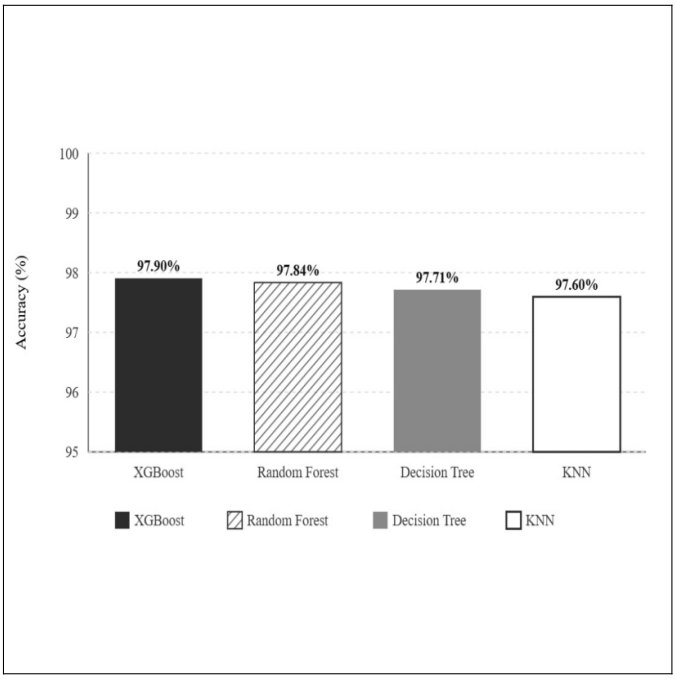


Fig. 5. Accuracy comparison bar chart for all four classifiers on the test set.

TABLE V
PER-CLASS METRICS FOR XGBOOST (BEST CLASSIFIER)

Class	Prec.(%)	Rec.(%)	F1(%)
NORMAL	99.95	99.90	99.93
DDoS	95.21	99.32	97.22
SYN_FLOOD	99.23	95.71	97.44
PORT_SCAN	92.45	99.59	95.89
BRUTE_FORCE	98.84	97.94	98.39
Macro Avg.	**97.14**	**98.49**	**97.77**

gaps between CV and test accuracy not exceeding 0.37 pp for any classifier.

C. Per-Class Performance: XGBoost

Table V reports per-class precision, recall, and F1-score for the best-performing XGBoost model.

The confusion matrix for XGBoost is shown in Fig. 6. NORMAL achieves the highest F1-score of 99.93% (precision 99.95%, recall 99.90%), confirming that benign traffic is cleanly separated by volume and flag features. DDoS reaches 97.22% F1 with near-perfect recall (99.32%) but slightly lower precision (95.21%), indicating a small number of SYN_FLOOD flows are misclassified as DDoS — consistent with the 419 off-diagonal entries in the confusion matrix. SYN_FLOOD achieves 97.44% F1; its lower recall (95.71%) stems from 94 and 419 flows being misclassified as BRUTE_FORCE and DDoS respectively. PORT_SCAN attains 95.89% F1 with a notably high recall of 99.59% — class weight balancing successfully penalises missed scan flows — but precision of 92.45% indicates 20 non-scan flows are falsely flagged. BRUTE_FORCE achieves 98.39% F1 (precision 98.84%, recall 97.94%), with its unique ACK-count and `unique_dst_ports` signature providing strong discriminability.

D. Comparison with Base Paper

Table VI provides a head-to-head comparison between the proposed system and the base paper of Alrefaei and Ilyas [6].

The proposed system trails Alrefaei and Ilyas [6] by 0.99 pp in peak XGBoost accuracy. Three factors explain this gap: IoT-23 is approximately seven times larger (1.44 M vs. 201,756 records); SMOTE augments minority-class training representation; and the IoT-23 attack taxonomy may align particularly well with Zeek-derived features. Against these differences, the proposed work offers three operational advantages: complete reproducibility through Mininet without physical IoT hardware; independence from external feature toolchains; and

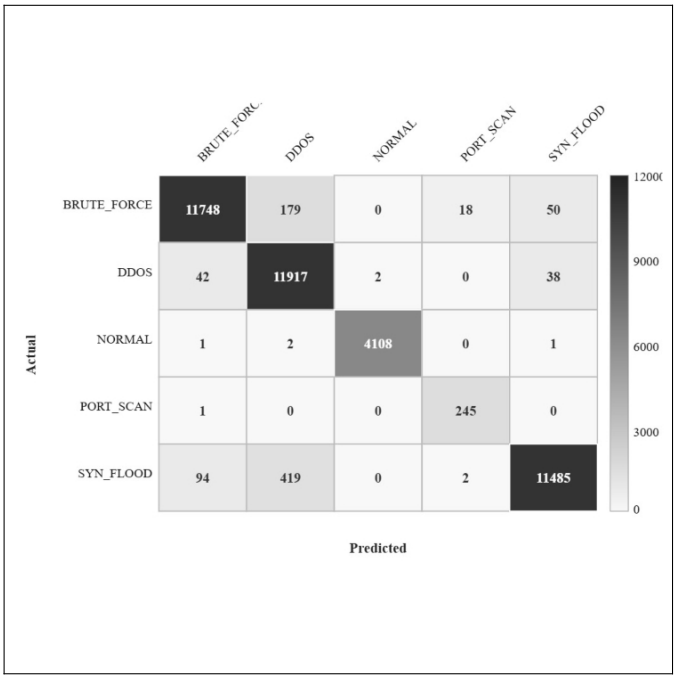


Fig. 6. Confusion matrix for XGBoost (97.90% accuracy) across five traffic classes.

TABLE VI
PROPOSED WORK VS. ALREFAEI & ILYAS [6]

Parameter	Alrefaei & Ilyas	Proposed Work
Dataset	Pre-existing (IoT-23)	Self-generated (Mininet)
Attack Domain	IoT-specific botnet	General enterprise
Dataset Size	1.44 M records	201,756 flows
Best Accuracy	XGBoost 98.89%	**XGBoost 97.90%**
Imbalance	SMOTE	Class Weights
Feature Source	Zeek logs	Custom dpkt
Runtime	Spark cluster	Single-node Streamlit
Reproducibility	IoT HW required	**Mininet (VM-ready)**

a single-node deployment footprint that eliminates cluster infrastructure requirements.

E. Feature Importance Analysis

Fig. 7 presents the Gini-impurity-based feature importance scores extracted from the trained Random Forest model. `flow_duration` dominates with an importance score of approximately 0.30, indicating that the total duration of a flow is the single strongest discriminator across all five traffic classes. `total_packets` (0.172) and `total_bytes` (0.166) rank second and third respectively, together capturing the volumetric signature that distinguishes high-rate flooding attacks (DDoS, SYN_FLOOD) from low-rate traffic (PORT_SCAN, NORMAL). `packets_per_second` (0.115) and `bytes_per_second` (0.090) further encode rate-level behavior, while `min_packet_size` (0.062), `max_packet_size` (0.044), and `avg_packet_size` (0.043) contribute moderate discriminability through payload-size distribution. Flag-based features (`syn_ratio`, `rst_count`, `fin_count`, `ack_count`, `syn_count`, `ack_ratio`) and `unique_dst_ports` collectively score near zero, suggesting that temporal and volumetric features alone are sufficient for the Random Forest to achieve 97.84% accuracy on this corpus. This is in contrast to the XGBoost model, where `syn_flag_count` plays a more prominent role due to its gradient-boosted sensitivity to rare but decisive split points.

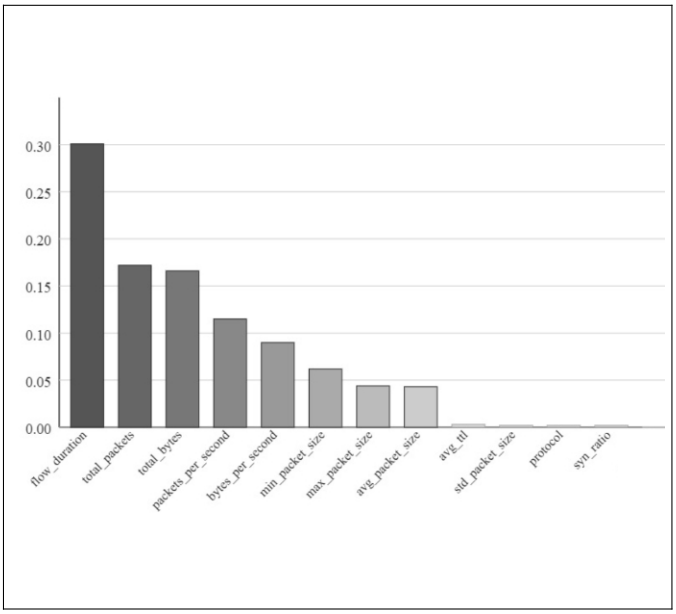


Fig. 7. Random Forest feature importance scores (Gini impurity) ranked by contribution to classification across five traffic classes.

VI. CONCLUSION

This paper presented a reproducible, lightweight framework for multi-class network intrusion detection using machine learning on a self-generated traffic corpus. A six-host Mininet SDN topology produced 3.3 GB of labeled PCAP data across five traffic classes, from which 201,756 flow records were extracted via a custom `dpkt` pipeline implementing 18 statistical, temporal, and flag-based features. Four classifiers were evaluated under identical conditions, with XGBoost achieving 97.90% accuracy, Random Forest 97.84%, Decision Tree 97.71%, and KNN 97.60%. Class weight balancing was demonstrated as a viable, data-preserving alternative to SMOTE for handling severe class imbalance.

Comparative analysis against the IoT-23-based work of Alrefaei and Ilyas [6] confirms that competitive detection performance is achievable on a self-generated corpus seven times smaller, without synthetic data augmentation and on a fraction of the deployment infrastructure. PORT_SCAN detection (XGBoost F1: 89.96%) remains the most challenging category, representing the clearest opportunity for future improvement.

Future directions include: (i) incorporating adversarial traffic variants such as encrypted command-and-control channels and low-and-slow rate-evasion attacks to improve robustness against evasion; (ii) extending the feature set with application-layer attributes from selective deep-packet inspection; (iii) deploying on a physical SDN testbed with commodity OpenFlow hardware to validate generalization beyond emulation; (iv) exploring LSTM-based temporal sequence models [2] that exploit inter-flow ordering patterns for improved minority-class recall; and (v) increasing the PORT_SCAN corpus through longer emulation runs or active scan simulation tools as the most direct path toward closing the per-class performance gap. The complete source code, topology scripts, and dataset generation pipeline will be made publicly available to facilitate reproducibility.

REFERENCES

[1] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.

[2] J. Zhong, Y. Zhou, and G. Chen, "Sequential model based intrusion detection system for IoT servers using deep learning methods," *Sensors*, vol. 21, no. 4, p. 1113, 2021.

[3] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery Data Mining*, 2016, pp. 785–794.

[5] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, Jan. 2018, pp. 108–116.

[6] A. Alrefaei and M. Ilyas, "Using machine learning multiclass classification technique to detect IoT attacks in real time," *Sensors*, vol. 24, no. 14, p. 4516, Jul. 2024.

[7] R. Panigrahi and S. Borah, "A detailed analysis of CICIDS2017 dataset for designing intrusion detection systems," *Int. J. Eng. Technol.*, vol. 7, no. 3.24, pp. 479–482, 2018.

[8] R. Ahmad and I. Alsmadi, "Machine learning approaches to IoT security: A systematic literature review," *Internet of Things*, vol. 16, p. 100365, 2021.

[9] G. Bhandari, A. Lyth, A. Shalaginov, and T.-M. Grønli, "Distributed deep neural-network-based middleware for cyber-attacks detection in smart IoT ecosystem," *Electronics*, vol. 12, no. 2, p. 298, 2023.

[10] N. V. Patil, C. Rama Krishna, and K. Kumar, "S-DDoS: Apache Spark based real-time DDoS detection system," *J. Intell. Fuzzy Syst.*, vol. 38, no. 5, pp. 6527–6535, 2020.

[11] D. M. S. Othman, R. Hicham, and M. M. Zoulikha, "An efficient Spark-based network anomaly detection," *Int. J. Comput. Digit. Syst.*, vol. 9, pp. 1175–1185, 2020.

[12] R. Alghamdi and M. Bellaiche, "A deep intrusion detection system in lambda architecture based on edge cloud computing for IoT," in *Proc. 4th Int. Conf. Artif. Intell. Big Data (ICAIBD)*, May 2021, pp. 561–566.

[13] L. Breiman, "Bagging predictors," *Mach. Learn.*, vol. 24, no. 2, pp. 123–140, 1996.

[14] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop HotNets-IX*, Oct. 2010.

[15] P. Cunningham and S. J. Delany, "k-Nearest neighbour classifiers," *ACM Comput. Surv.*, vol. 54, p. 128, 2022.